published by a particular state education department (http://mng.bz/p6EK). It is unclear why they were included or how—perhaps as HTML comments, because their exposure doesn't appear to be deliberate. Without wanting to prejudice any potential legal proceedings, this should serve as a reminder that comments in HTML source code are not private, despite the fact that browsers do not display them. If you are ever tempted to include information that is not intended for display in a Razor file, use Razor comments (`@* ... *@`) to prevent it from being rendered. The best practice is not to include anything confidential within content that might be exposed to users at all.

When we covered authorization, we looked at authorizing resources (section 10.5) as well as endpoints. Resource-based authorization is a best practice. It enforces record ownership rather than accepting that users can perform read, update, and delete on any record. It also mitigates against URL tampering, which might expose records the user is not entitled to access.

We also looked at using the `Authorize` attribute to apply authorization policies on a page-by-page basis. I recommended centralizing your policies, so they are easier to review and manage. This chapter provides a good opportunity to reinforce that advice. If you find access control review a time-consuming chore, you are less likely to do it as often as you should, which is more likely result in broken authorization.

13.2.2 Cross-site request forgery

Cross-site request forgery, or CSRF, exploits a vulnerability that causes an authenticated user to make an involuntary request to a website. The request could result in a command being processed by the website in question that the user did not intend, such as changing a password or transferring money.

To illustrate this, imagine you visit your banking website at mysavings.com. You log in, and the banking site adds an authentication cookie to your browser. The cookie remains valid for whatever term the banking site specified. While the cookie is valid, you visit another site, spoofsite.com, perhaps as a result of clicking an interesting looking link in a phishing email. The page you visit includes code that generates a post request to mysavings.com that results in some action being performed on your account. Landing on the page in spoofsite.com triggers the code, and the request is sent. This is the *cross-site* request. The banking site processes the request because it is accompanied by your authentication cookie, which tells your bank site you are currently logged in (figure 13.4).